TESTNG CLASS 5

Complete Notes

Assertions • SoftAssert • Retry Analyzer • Annotation Transformer • Listeners

Beginner-friendly explanations, commented Java examples, real testing scenarios, best practices, and interview revision.

BASED ON YOUR UPLOADED CLASS MATERIAL

Main learning path

- Understand hard assertions and soft assertions, including the purpose of assertAll().
- Build an assertion strategy for Selenium UI checks and API validations.
- Understand TestNG retry behavior with IRetryAnalyzer.
- Use IAnnotationTransformer to apply retry logic selectively or across a suite.
- Learn where ITestListener and other TestNG extension interfaces fit in a framework.

Important learning note

The uploaded examples include a few syntax errors and simplified explanations. This guide preserves the topics and intent, then shows corrected Java/XML and calls out practical caveats so the examples are safe to learn from.

How to Use These Notes

Read the assertions sections first, then move to retry behavior and listener registration. The examples use TestNG with Java. Selenium snippets assume Selenium WebDriver and a browser driver are configured in your project; they are teaching examples, not a ready-to-run project scaffold.

Contents / learning roadmap

- 1. Test assertions: what they check and why they matter
- 2. Hard assertions with org.testng.Assert
- 3. Soft assertions with org.testng.asserts.SoftAssert
- 4. Hard vs. soft: choose the right tool for the scenario
- 5. Real-world examples: UI and API validation
- 6. Common assertion mistakes and best practices
- 7. TestNG listeners and extension interfaces
- 8. IRetryAnalyzer: bounded retry of failed tests
- 9. IAnnotationTransformer: applying retry logic centrally
- 10. ITestListener: reacting to test events
- 11. Putting it together in a maintainable framework
- 12. Troubleshooting, exercises, interview questions, and glossary

Learning outcomes

- Explain actual vs. expected values and write clear assertions.
- Predict which lines execute after a hard assertion fails.
- Use SoftAssert for multiple related validations and always call assertAll().
- Explain what retry() returning true or false means.
- Calculate total attempts correctly when a retry limit is configured.
- Register a transformer in testng.xml and understand when not to use global retries.

> **Terminology**
>
> **Verify** is commonly used informally to mean a soft assertion. In TestNG, the API you will use is SoftAssert; it is not a separate annotation called @Verify.

1. What Is an Assertion?

An assertion compares an actual result from the application with an expected result defined by the test. It turns a test from “perform these steps” into “perform these steps and prove that the result is correct.”

Example: After logging in, a test should confirm that the dashboard title is displayed. Clicking the Login button alone does not prove that login succeeded.

A simple assertion

```
String actualTitle = driver.getTitle();
String expectedTitle = "Dashboard";
Assert.assertEquals(actualTitle, expectedTitle,
      "The dashboard title should match after login");
```

Actual value vs. expected value

Term	Meaning	Example
Actual	The value observed during the test run.	driver.getTitle() returns Dashboard.
Expected	The value the requirement says should be observed.	The specification says title must be Dashboard.
Assertion result	Comparison between actual and expected.	Equal means pass; mismatch means assertion failure.

What happens after an assertion?

TestNG reports a test method as passed when it finishes without an uncaught failure. If a hard assertion fails, it throws an assertion error. The remaining statements in that same test method are skipped, but TestNG can still continue with other independent test methods according to the suite configuration.

> **Keep these two scopes separate**
>
> A failed assertion usually stops the current test method; it does not automatically stop the entire TestNG suite. A configuration failure (for example, browser setup failure) may affect dependent tests differently.

2. Hard Assertions: org.testng.Assert

A hard assertion is the normal TestNG assertion provided by org.testng.Assert. If it fails, execution of the current test method stops at that line because an assertion error is thrown.

2.1 Correct syntax

Example 1 - passing hard assertion

```
import org.testng.Assert;
import org.testng.annotations.Test;

public class HardAssertExample {
   @Test
   public void verifyUserCount() {
      System.out.println("Test started");

      int actualCount = 12;
      int expectedCount = 12;
      Assert.assertEquals(actualCount, expectedCount,
            "User count did not match");

      System.out.println("This line executes when the assertion passes");
   }
}
```

2.2 Line-by-line explanation

- import org.testng.Assert; imports TestNG assertion methods.
- @Test tells TestNG that the method is a test method.
- actualCount stores the observed result; expectedCount stores the expected result.

- Assert.assertEquals(actualCount, expectedCount, message) passes when values match and reports the message when they do not.
- The final println runs only if the assertion does not throw an error.

2.3 What if the assertion fails?

Example 2 - failing hard assertion

```
@Test
public void verifyIncorrectCount() {
   System.out.println("Test started");

   Assert.assertEquals(12, 100,
         "Expected 100 users but observed 12");

   System.out.println("This line will NOT execute");
}
```

Expected result: verifyIncorrectCount is marked failed and the last print statement is skipped. Another independent @Test method may still run.

2.4 Common assertion methods

Method	Use it to check	Example
assertEquals(actual, expected)	Equality of values.	Assert.assertEquals(statusCode, 200);
assertNotEquals(actual, expected)	Values should differ.	Assert.assertNotEquals(message, "");
assertTrue(condition)	A condition must be true.	Assert.assertTrue(button.isDisplayed());
assertFalse(condition)	A condition must be false.	Assert.assertFalse(errorBanner.isDisplayed());
assertNotNull(value)	A value must exist.	Assert.assertNotNull(accessToken);
assertNull(value)	A value must be null.	Assert.assertNull(optionalField);
fail(message)	The test must fail at this point.	Assert.fail("Unexpected branch");

> **Order of parameters**
>
> In the examples here, TestNG assertEquals is written as **actual first, expected second**. Use a clear failure message to make reports easier to diagnose.

3. Soft Assertions: org.testng.asserts.SoftAssert

A soft assertion records a failed check but does not immediately stop the test method. This is useful when you want to check several independent properties on the same screen or response and report all mismatches together.

3.1 Correct syntax and the essential assertAll()

Example 3 - multiple soft assertions

```
import org.testng.annotations.Test;
import org.testng.asserts.SoftAssert;

public class SoftAssertExample {
   @Test
   public void verifyProfileSummary() {
      SoftAssert soft = new SoftAssert();

      soft.assertEquals("Saikiran", "Saikiran",
            "First name should match");
      soft.assertEquals("Premium", "Standard",
            "Account type should match");
      soft.assertTrue(5 > 10,
            "The sample condition should be true");

      // Required: report any failures collected above.
      soft.assertAll();
   }
}
```

3.2 What each line does

- new SoftAssert() creates a collector for this test method.
- Each soft.assert... evaluates a check. If it fails, the failure is recorded and execution continues.
- soft.assertAll() checks whether any failures were collected. If yes, it throws an assertion error and TestNG marks the test failed.
- The example above intentionally contains two failed checks. It continues through all checks, then fails at assertAll() and reports the collected failures.

> **Critical rule: never forget assertAll()**
>
> If a soft assertion fails but assertAll() is never called, TestNG may mark the test as passed even though a recorded check failed. That creates a false-positive result.

3.3 Incorrect pattern - no assertAll()

Do not copy this pattern

```
@Test
public void incorrectSoftAssertUsage() {
   SoftAssert soft = new SoftAssert();
   soft.assertEquals(12, 100, "Count should match");

   System.out.println("This line executes");
   // Missing soft.assertAll();
}
```

The method can complete without TestNG being told to throw the recorded soft-assertion failures. Always include assertAll() after the related validations.

3.4 More realistic soft assertion example

Product card: validate independent UI properties

```
@Test
public void verifyProductCard() {
   SoftAssert soft = new SoftAssert();

   String actualName = "Wireless Mouse";
   String actualPrice = "$19.99";
   boolean imageVisible = true;
   boolean addToCartVisible = false;

   soft.assertEquals(actualName, "Wireless Mouse",
         "Product name should be correct");
   soft.assertEquals(actualPrice, "$19.99",
         "Product price should be correct");
   soft.assertTrue(imageVisible,
         "Product image should be visible");
   soft.assertTrue(addToCartVisible,
         "Add to Cart button should be visible");

   soft.assertAll();
}
```

This is a reasonable soft-assert use case: the name, price, image visibility, and button visibility can all be inspected in one page visit. If the button is missing, the other useful validations still run.

4. Hard Assert vs. Soft Assert

Aspect	Hard assertion (`Assert`)	Soft assertion (`SoftAssert`)
On failure	Throws immediately; remaining statements in the current test method are skipped.	Records failure and allows the method to continue.
When result fails	Failure is reported at the assertion line.	Failure is reported when assertAll() is called.
Required final step	No special finalization call.	assertAll() is required after the checks.
Best fit	A prerequisite or critical condition.	Several independent checks that are useful to collect together.
Risk	Can stop a useful diagnostic sequence early.	Can produce false passes if assertAll() is omitted.
Example	If login did not succeed, stop before checkout.	Check title, product name, price, image, and button on one product page.

How to choose in an actual test

Use a hard assertion when the next action is not safe or meaningful unless the current condition passed. Example: if login failed, do not continue to checkout.

Use soft assertions when later checks are independent and still provide diagnostic value. Example: validate several labels in a profile panel in one run.

Avoid making every assertion soft

Soft assertions are not automatically better. A sequence of dependent actions often needs a hard assertion at each critical boundary; otherwise the test may continue in an invalid state and create confusing failures.

5. Real-World Examples

5.1 Selenium UI example - login prerequisite and dashboard checks

Scenario: after submitting valid login credentials, first confirm that the dashboard loaded. That is a prerequisite and should be a hard assertion. Once the dashboard is confirmed, verify independent widgets with soft assertions.

Illustrative Selenium code - replace URL and locators

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.Test;
import org.testng.asserts.SoftAssert;

public class DashboardValidationTest {
   @Test
   public void validateDashboardWidgets() {
      WebDriver driver = new ChromeDriver();
      try {
         driver.manage().window().maximize();
         driver.get("https://example.test/dashboard");

         // Replace example.test and locators with your application values.
         WebElement dashboard = driver.findElement(By.id("dashboard"));
         Assert.assertTrue(dashboard.isDisplayed(),
               "Dashboard did not load; stop this test here");

         SoftAssert soft = new SoftAssert();
         soft.assertTrue(driver.findElement(By.id("welcome-message")).isDisplayed(),
               "Welcome message should be visible");
         soft.assertTrue(driver.findElement(By.id("orders-widget")).isDisplayed(),
               "Orders widget should be visible");
         soft.assertTrue(driver.findElement(By.id("profile-widget")).isDisplayed(),
               "Profile widget should be visible");

         soft.assertAll();
      } finally {
         driver.quit();
      }
   }
}
```

What this example teaches

- The hard assertion protects the test from checking widgets on a page that never loaded.
- The soft assertions collect independent dashboard widget checks.
- finally runs cleanup even if an assertion or locator fails, so the browser session is not intentionally left open.
- In a framework, browser setup/teardown usually belongs in @BeforeMethod and @AfterMethod or a shared base test; avoid duplicating setup in every test.

5.2 API validation example - collect independent response checks

Scenario: a GET endpoint returns an employee record. The HTTP status is a prerequisite; if it is not successful, response-field checks may not be meaningful. After a successful status is confirmed, independent fields can be checked together.

API-style validation flow without a specific client dependency

```
// Illustrative TestNG assertion flow; actualResponse comes from your API client.
int statusCode = 200;
String employeeName = "Asha Rao";
String department = "QA";
boolean employeeIdPresent = true;

Assert.assertEquals(statusCode, 200,
      "Employee endpoint should return HTTP 200");

SoftAssert soft = new SoftAssert();
soft.assertEquals(employeeName, "Asha Rao", "Employee name should match");
soft.assertEquals(department, "QA", "Department should match");
soft.assertTrue(employeeIdPresent, "Employee ID should be present");
soft.assertAll();
```

The values are illustrative placeholders. In a real Rest Assured test, assign the status and body fields from the actual response instead of hardcoding them.

6. Assertion Best Practices and Common Mistakes

Best practices

- Assert business requirements, not implementation details that may change without affecting behavior.
- Use meaningful assertion messages such as Cart count should be 1 after adding one item.
- Keep assertions near the action or state they validate so a failure is easy to diagnose.
- Use hard assertions for prerequisites and soft assertions for independent checks.
- Create one SoftAssert instance per test method and call assertAll() once all checks are complete.
- Avoid sharing a SoftAssert as a static field across test methods; it can mix failures between tests.
- For decimals, use an appropriate tolerance/delta instead of exact floating-point equality.
- Do not catch and ignore AssertionError; that turns a real failure into a misleading pass.
- Keep test data predictable and independent so assertion failures point to application defects rather than stale state.

Common mistakes

Mistake	Why it is a problem	Better approach
Wrong actual/expected order	Failure output becomes confusing.	Use a consistent actual, expected, message pattern.
SoftAssert without assertAll()	Recorded failures can be lost and the test may pass.	Call assertAll() at the end of the test.
One huge test with unrelated checks	A single failure obscures which feature broke.	Split unrelated behaviors into separate test methods.
Soft assert after a failed prerequisite	Later checks run against an invalid state.	Use a hard assertion for prerequisites such as successful login.
Assertions that only check not-null	A value may exist but still be wrong.	Validate type, exact business value, range, and relevant invariants.

Mistake	Why it is a problem	Better approach
Ignoring failure messages	CI logs become harder to understand.	Add concise, business-specific messages.

7. What Is a TestNG Listener?

A listener is an extension point that lets your framework react to events during a TestNG run. Examples include a test starting, passing, failing, being skipped, or a suite starting and finishing.

The source notes group several TestNG interfaces under "listeners." For precision, ITestListener, ISuiteListener, IConfigurationListener, IMethodInterceptor, and IReporter serve different extension purposes; IRetryAnalyzer is specifically a retry decision interface, not the same kind of test-event callback as ITestListener.

Interface	Main purpose	Common use
ITestListener	Receive test-method events.	Log pass/fail/skip and capture failure context.
IRetryAnalyzer	Decide whether a failed test is retried.	Retry a test within a bounded policy.
IAnnotationTransformer	Modify test annotations before execution.	Attach a retry analyzer or adjust metadata centrally.
ISuiteListener	React when a suite starts or ends.	Create suite-level reporting or cleanup.
IConfigurationListener	Observe configuration method results.	Diagnose failures in setup/teardown methods.
IMethodInterceptor	Filter or reorder discovered test methods.	Custom selection or ordering rules.
IReporter	Generate a report after the run.	Build a consolidated custom report.

Why listeners are useful in a framework

- Consistent pass/fail logging across hundreds of tests.
- A central place to trigger screenshot capture on test failure (with an appropriately designed driver manager).
- Adding metadata, links, or environment information to reports.
- Avoiding repeated reporting code in every test method.

8. IRetryAnalyzer - Retry a Failed Test

IRetryAnalyzer is a TestNG interface with a retry(ITestResult result) method. Return true when TestNG should run the failed test again; return false when the retry budget is exhausted or retrying is not appropriate.

> **Retries do not repair a test**
>
> A retry is a diagnostic/reliability mechanism for transient flakiness. It should not hide a repeatable product defect. Keep the initial failure and retry history visible in CI reports.

8.1 Simple bounded retry analyzer

Example - initial attempt plus at most two retries

```
package com.example.listeners;

import org.testng.IRetryAnalyzer;
import org.testng.ITestResult;

public class RetryAnalyzer implements IRetryAnalyzer {
   private int retryCount = 0;
   private static final int MAX_RETRIES = 2;

   @Override
   public boolean retry(ITestResult result) {
      if (retryCount < MAX_RETRIES) {
         retryCount++;
         System.out.println("Retrying " + result.getName()
               + " - retry " + retryCount + "/" + MAX_RETRIES);
         return true;
      }
      return false;
   }
}
```

8.2 Explanation of the code

- implements IRetryAnalyzer tells TestNG that this class supplies retry decisions.
- retryCount tracks how many retries have been requested for this analyzer instance.
- MAX_RETRIES = 2 permits at most two retries after the first attempt.
- The first failure causes retry() to return true, so TestNG schedules another attempt.
- Once retryCount reaches 2, the next failure returns false; TestNG stops retrying that failure.

Attempt	When it occurs	Retry result
1	Original execution	Not a retry decision yet; a failure invokes retry().
2	Retry #1	retryCount becomes 1; return true.
3	Retry #2	retryCount becomes 2; return true.
After attempt 3 fails	Retry budget exhausted	Return false; no fourth attempt.

> **Important counting rule**
>
> MAX_RETRIES = 2 means **up to 3 total attempts**: 1 original attempt + 2 retries. It does not mean only two total executions.

8.3 Attach retry logic to a specific test

Attach retry behavior through @Test

```
import org.testng.Assert;
import org.testng.annotations.Test;
import com.example.listeners.RetryAnalyzer;

public class PaymentTest {
   @Test(retryAnalyzer = RetryAnalyzer.class)
   public void verifyPaymentGatewayResponse() {
      // Call the test environment's payment API or UI here.
      int actualStatus = 503; // Illustrative temporary server response.
      Assert.assertEquals(actualStatus, 200,
            "Payment gateway should return HTTP 200");
   }
}
```

The test above deliberately demonstrates the mechanics; if it always receives 503, it will fail after the retry budget. A retry cannot make a broken service healthy.

8.4 Practical retry policy

The simple example retries every failure. In a production framework, consider the failure cause before retrying. A short-lived browser startup issue or intermittent infrastructure timeout may justify a retry. A deterministic wrong price, wrong business rule, or repeatable assertion mismatch usually needs investigation instead of repeated execution.

- Keep retries bounded (often one or two retries rather than an unlimited loop).
- Log the method name, exception, retry count, environment, and browser/build details.
- Do not convert “failed first time, passed on retry” into an invisible success; report it as flaky.
- Do not retry destructive tests without checking idempotency. Retrying “create payment” can duplicate side effects.
- If runs are parallel, ensure the retry counter is not unintentionally shared across different tests. Verify analyzer lifecycle behavior for the TestNG version and parallel configuration you use.

9. IAnnotationTransformer - Configure Retry Centrally

Applying retryAnalyzer = ... on every test works for a small suite. An IAnnotationTransformer can attach a retry analyzer before TestNG executes selected test methods, which reduces repeated annotation changes. It should be applied selectively rather than enabling retries indiscriminately.

9.1 Transformer example

Selective transformer - use your actual package names

```
package com.example.listeners;

import java.lang.reflect.Constructor;
import java.lang.reflect.Method;

import org.testng.IAnnotationTransformer;
import org.testng.annotations.ITestAnnotation;

public class RetryTransformer implements IAnnotationTransformer {
  @Override
  public void transform(ITestAnnotation annotation,
                        Class testClass,
                        Constructor testConstructor,
                        Method testMethod) {
    // Example policy: only methods whose names start with "flaky"
    // receive automatic retry behavior.
    if (testMethod != null
          && testMethod.getName().startsWith("flaky")) {
      annotation.setRetryAnalyzer(RetryAnalyzer.class);
    }
  }
}
```

9.2 How the transformer works

- transform() is called while TestNG processes annotations.
- ITestAnnotation annotation represents the @Test metadata that TestNG is reading.
- testMethod identifies the test method when the annotation is method-based; check for null before using it.
- annotation.setRetryAnalyzer(RetryAnalyzer.class) tells TestNG to use your retry analyzer for that selected method.
- The sample uses names beginning with flaky as a simple selection rule. In a real project, prefer an explicit documented convention or other maintainable selection strategy.

9.3 Register the transformer in testng.xml

Valid XML structure - replace package/class names

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Regression Suite">
  <listeners>
    <listener class-name="com.example.listeners.RetryTransformer"/>
  </listeners>
  <test name="UI Regression">
    <classes>
      <class name="com.example.tests.PaymentTest"/>
    </classes>
  </test>
</suite>
```

> **Registration note**
>
> Register an IAnnotationTransformer using testng.xml or programmatic TestNG configuration so it is active before TestNG processes test annotations. Avoid registering the transformer only with the test-class @Listeners annotation; TestNG documentation warns that it is too late for annotation transformation.

When to use per-test vs. global configuration

Option	Use when	Watch out for
@Test(retryAnalyzer = ...)	Only a few known tests need retries.	The annotation must be maintained on each applicable method.
IAnnotationTransformer	A consistent suite policy should attach retry behavior centrally.	A broad transformer can accidentally retry deterministic failures and hide real defects.

10. ITestListener - Observe Test Events

ITestListener is useful for logging when tests start, pass, fail, or are skipped. It does not itself decide whether a test should be retried; that is the role of IRetryAnalyzer. A listener can, however, log that a test failed and record useful diagnostic details.

Common event hooks; add other callbacks required by your TestNG version

```
package com.example.listeners;

import org.testng.ITestListener;
import org.testng.ITestResult;

public class TestRunListener implements ITestListener {
    @Override
    public void onTestStart(ITestResult result) {
        System.out.println("START: " + result.getName());
    }

    @Override
    public void onTestSuccess(ITestResult result) {
        System.out.println("PASS: " + result.getName());
    }

    @Override
    public void onTestFailure(ITestResult result) {
        System.err.println("FAIL: " + result.getName());
        if (result.getThrowable() != null) {
            System.err.println(result.getThrowable().getMessage());
        }
    }

    @Override
    public void onTestSkipped(ITestResult result) {
        System.out.println("SKIP: " + result.getName());
    }

    @Override
    public void onTestFailedButWithinSuccessPercentage(ITestResult result) {
        System.out.println("FAILED WITHIN SUCCESS PERCENTAGE: " + result.getName());
    }
}
```

Depending on the TestNG version and interfaces used, you may also need to implement additional listener methods or can rely on default implementations. Check the API for the exact version configured in your project.

Register the listener

XML registration

```
<suite name="Regression Suite">
  <listeners>
    <listener class-name="com.example.listeners.TestRunListener"/>
  </listeners>
  <test name="Regression">
    <classes>
      <class name="com.example.tests.PaymentTest"/>
    </classes>
  </test>
</suite>
```

ITestListener vs. IRetryAnalyzer vs. IAnnotationTransformer

Question	Interface	Answer
Who reports test events?	ITestListener	“This test started / passed / failed / was skipped.”
Who decides to retry?	IRetryAnalyzer	“Return true to retry this failed test.”
Who attaches retry policy before execution?	IAnnotationTransformer	“Set the retry analyzer on selected @Test annotations.”

11. Putting the Concepts Together

A practical automation framework may use these features together, but each component should have one responsibility:

1. A test method performs actions and checks the application with hard or soft assertions.
2. A retry analyzer decides whether a failed method should be attempted again within a strict retry limit.
3. An annotation transformer attaches the retry analyzer to only the intended tests.
4. An ITestListener logs test lifecycle events and adds diagnostics to test reports.
5. The reporting layer should preserve the first failure and show retries; do not report a retried flaky test as if it had passed cleanly on the first attempt.

Suggested package structure

Illustrative project organization

```
src/test/java/
 com/example/tests/
   LoginTest.java
   ProductTest.java
   PaymentTest.java
 com/example/listeners/
   RetryAnalyzer.java
   RetryTransformer.java
   TestRunListener.java
 com/example/base/
   BaseTest.java
 testng.xml
```

A sample execution flow

1. TestNG reads the suite XML and registers listeners/transformers. 2. The transformer changes retry metadata for selected methods. 3. A test method runs and an assertion fails. 4. ITestListener receives the failure event and logs diagnostics. 5. IRetryAnalyzer is asked whether to retry. 6. If it returns true, another attempt runs; if it returns false, the failed result remains. 7. The final report must make clear whether the test failed immediately, passed after retry, or exhausted the retry budget.

12. Troubleshooting Checklist

Symptom	Likely cause	What to check
Soft assertion failure not in report	assertAll() is missing or unreachable.	Confirm soft.assertAll() executes after the assertions.
Test passes despite wrong value	A SoftAssert was used without calling assertAll().	Add the final aggregation call and rerun.
Lines after assertion do not run	A hard assertion failed.	Inspect actual/expected values and the failure message.
Test is not retried	Analyzer not attached, no retries left, or test not eligible.	Check @Test(retryAnalyzer=...), transformer registration, and retry counter.
All tests unexpectedly retry	Transformer applies analyzer to every test.	Narrow the transformer selection logic.
Transformer appears not to run	It is registered too late or XML class name is wrong.	Register through testng.xml and verify the fully qualified class name.
Retries obscure a real bug	Retry policy treats all failures identically.	Limit retries, classify causes, and surface flaky results.
Duplicate records after retry	A non-idempotent test was repeated after a side effect.	Use unique test data, cleanup, or do not retry that operation blindly.

13. Hands-On Exercises

Complete these before checking the solution hints. They intentionally start simple and build toward a mini-framework.

6. Write a hard assertion that confirms an actual cart count of 2 equals the expected count of 2, with a failure message.
7. Write three soft assertions for profile name, email visibility, and profile icon visibility. Ensure the test fails if any check fails.
8. Predict which print statements execute when Assert.assertEquals(10, 20) is followed by two print statements.
9. Create a RetryAnalyzer that permits one retry after the original attempt. How many total attempts are possible?
10. Modify RetryTransformer so that only test methods whose names begin with flaky receive the retry analyzer.
11. Implement an ITestListener callback that logs the test name on failure.
12. Explain why retrying a failed payment creation call without idempotency protection can be dangerous.

Solution hints

- Assert.assertEquals(actualCartCount, 2, "Cart count should be 2");

- Use one SoftAssert, call soft.assertEquals/soft.assertTrue for all three checks, then call soft.assertAll().
- Only the code before the hard assertion executes; statements after the failed assertion in the same test method are skipped.
- One retry means at most two total attempts: one original run plus one retry.
- Check testMethod != null && testMethod.getName().startsWith("flaky") before calling annotation.setRetryAnalyzer(...).
- Override onTestFailure(ITestResult result) and inspect result.getName() and result.getThrowable().
- The first attempt may have created the payment even if the client timed out; retry could create a second payment.

14. Interview Questions and Quick Answers

What is an assertion?

A comparison between an observed result and the expected behavior.

What is a hard assertion?

A TestNG assertion that throws immediately on failure and stops the current test method at that point.

What is a soft assertion?

A SoftAssert check that collects failures and reports them when assertAll() is called.

Why is assertAll mandatory?

Without it, collected soft-assertion failures may never be surfaced as a test failure.

Can TestNG execute the next test after a hard assertion fails?

Usually yes for other independent test methods, while the remaining lines in the failing method are skipped.

What is IRetryAnalyzer?

A TestNG interface whose retry(ITestResult) method decides whether a failed test should be retried.

What does returning true from retry() mean?

Request another attempt for the failed test, subject to TestNG execution rules and the retry policy.

If max retries is 3, how many total attempts are possible?

Up to 4: the original attempt plus 3 retries.

What does IAnnotationTransformer do?

It allows annotation metadata, such as retry analyzer configuration, to be changed before test execution.

What does ITestListener do?

It receives test lifecycle events such as start, success, failure, and skip.

Should every failed test be retried?

No. Retries should be bounded and suited to known transient failure modes; otherwise they can hide product defects.

Why can a retry be risky for API/UI tests?

Repeated operations can create duplicate side effects or inconsistent test data unless the operation is idempotent or cleaned up.

15. Quick Reference

Need to...	Use	Remember
Stop the method on mismatch	Assert.assert...	Statements after a failure are skipped.
Collect several independent mismatches	SoftAssert	Call assertAll() after the checks.

Need to…	Use	Remember
Retry one known test	@Test(retryAnalyzer = RetryAnalyzer.class)	Use a bounded retry count.
Attach retry policy centrally	IAnnotationTransformer	Register it before annotations are processed; select tests carefully.
Log pass/fail/skip lifecycle events	ITestListener	Keep listener logic focused on diagnostics/reporting.
Investigate flakiness	Retry history + logs/screenshots	Do not erase the first failure or silently convert flaky to clean pass.

16. Glossary

Term	Definition
Assertion	A check that compares observed behavior with expected behavior.
Hard assertion	Assertion that throws immediately on failure.
Soft assertion	Assertion that stores failures until assertAll() reports them.
Flaky test	A test whose result varies across runs without a relevant change to code or test conditions.
Retry budget	Maximum number of retries allowed after the original attempt.
Listener	A TestNG extension interface that receives or participates in execution events.
Annotation transformer	A hook that changes annotation metadata before TestNG uses it.
Idempotent operation	An operation that can be repeated without creating additional side effects beyond the first application.

17. References and notes

The topics and sample flow are based on the uploaded file “TestNG class 5.txt”. Code has been rewritten with corrected Java syntax and expanded with practical examples. The following official documentation is useful for checking behavior against the TestNG version in your project:

- TestNG documentation: https://testng.org/documentation.html
- TestNG SoftAssert API: https://javadoc.io/doc/org.testng/testng/latest/org/testng/asserts/SoftAssert.html
- TestNG IRetryAnalyzer API: https://javadoc.io/doc/org.testng/testng/latest/org/testng/IRetryAnalyzer.html
- TestNG IAnnotationTransformer API: https://javadoc.io/doc/org.testng/testng/latest/org/testng/IAnnotationTransformer.html
- TestNG logging and results / listeners: https://testng.org/logging_and_results.html

> **Version note**
>
> Listener and transformer APIs can evolve between TestNG releases. Keep your examples aligned with the TestNG version declared in pom.xml; use the linked API documentation for that version if a method signature differs.